

PROJETO INTERDISCIPLINAR

Sistema de Gamificação para Plataforma de Educação Continuada

Engenharia de Software + Programação Orientada a Objetos

Betina Volpi • Gustavo Abreu • Icaro Dias • Rodrigo Neiland

Agenda

02

01

Contexto do projeto e problema

02

Product Backlog e Sprint Backlog

03

Arquitetura, domínio, DTOs, controllers e serviços

04

Swagger, testes de API, Docker e apresentação funcional

05

Bônus, limitações e próximos passos

Projeto Interdisciplinar

- Gamificação para engajamento em educação continuada.
- Problema: aumentar permanência de alunos em cursos EAD.
- Solução: API REST em Java/Spring Boot para usuários, cursos, matrículas e base de gamificação.
- Valor: benefícios progressivos, plano Premium, moedas e vouchers como evolução.

Product Backlog

1

Gestão de usuários e autenticação

2

Cadastro de alunos, professores e cursos

3

Matrícula de alunos em cursos

4

Progressão por conclusão de cursos

5

Plano Premium após 12 cursos

6

Moedas, vouchers e recompensas

Sprint Backlog

Fonte: Jira SXA Board

1

Sprint 1

Estrutura Spring Boot, Maven, domínio, entidades JPA e repositórios.

2

Sprint 2

Endpoints REST, DTOs, services, autenticação e Swagger.

3

Sprint 3

Docker, PostgreSQL, front-end, ajustes finais e documentação.



Inserir print/export do Jira: betinavolpi2006.atlassian.net/jira/software/projects/SXA/boards/34

Arquitetura e Clean Architecture

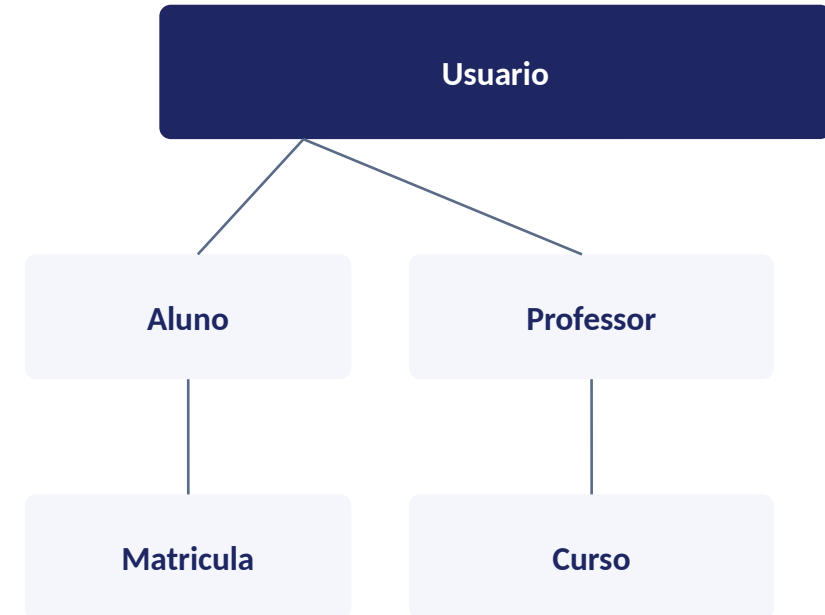
- Separação de responsabilidades e baixo acoplamento.
- Camadas: config, controller, domain, dto, repository e service.
- Evidência: `src/main/java/ananditos/sandraxandreia/`.



Camadas bem definidas facilitam teste, manutenção e expansão.

Arquitetura – Domínio

- Classes principais: Usuario, Aluno, Professor, Curso, Matricula, Assinatura.
- Usuario é classe base com herança JOINED para Aluno e Professor.
- Matricula liga Aluno e Curso; Professor possui Cursos.



• Evidências: *domain/usuario, domain/aluno, domain/professor, domain/curso, domain/matricula.*

DDD – Value Objects

UsuarioEmail

UsuarioCpf

UsuarioDataNascimento

UsuarioSenhaCriptografada

AlunoRA

ProfessorAreaFormacao



VOs encapsulam validação, significado de negócio e reduzem exposição de tipos primitivos.

Arquitetura – Repository

- Repositories abstraem persistência e estendem JpaRepository.
- Classes: AlunoRepository, CursoRepository, ProfessorRepository, UsuarioRepository, MatriculaRepository, AssinaturaRepository.
- Diferenciais: existsByEmailValor, existsByCpfValor e findByEmailValor.

● *A camada repository evita SQL manual para CRUD comum e concentra consultas específicas.*

JPA e Renderização das Tabelas

- JPA/Hibernate mapeia objetos Java para tabelas relacionais.
- Anotações: @Entity, @Table, @Id, @GeneratedValue, @OneToMany, @ManyToOne e @Inheritance.
- Bancos: H2 para desenvolvimento e PostgreSQL para execução com Docker.
- Evidências: pom.xml, application.properties, application-h2.properties, application-postgres.properties.



Evidências técnicas conectam o código à arquitetura apresentada.

Arquitetura – Service

- Services orquestram regras de aplicação.
- Classes: AlunoService, CursoService, ProfessorService, UsuarioService, MatriculaService.
- Responsabilidades: criar, listar, buscar, atualizar, deletar e validar dados.

● *Services traduzem intenções da API em operações de domínio e persistência.*

DTOs

AlunoRequestDTO**AlunoResponseDTO****CursoRequestDTO****CursoResponseDTO****ProfessorRequestDTO****ProfessorResponseDTO****MatriculaRequestDTO****MatriculaResponseDTO****UsuarioRequestDTO****UsuarioResponseDTO****LoginRequestDTO****LoginResponseDTO**

DTOs evitam expor entidades diretamente e controlam o contrato da API.

Controllers

Endpoint	Métodos
/aluno	GET, POST, PUT, DELETE
/curso	GET, POST, PUT, DELETE, PUT /{id}/status
/professor	GET, POST, PUT, DELETE
/matricula	GET, POST, PUT, DELETE
/usuario	GET, POST, PUT, DELETE
/login	POST

Swagger e Testes de API

- Swagger documenta e permite testar a API pelo navegador.
- Dependência: springdoc-openapi-starter-webmvc-ui no pom.xml.
- Configuração: springdoc.swagger-ui.path=/swagger-ui.html.
- Controllers usam @Tag e @Operation.
- Testes podem ser feitos via Swagger, Postman ou Insomnia.



Evidências técnicas conectam o código à arquitetura apresentada.

Deploy – Docker I

DOCKER

- Dockerfile com build multi-stage.
- Build: maven:3.9.12-eclipse-temurin-25.
- Runtime: eclipse-temurin:25-jre.
- Porta exposta: 8080.
- Execução: java -jar /app/app.jar.

Deploy – Docker Hub

- Docker Hub versiona e distribui imagens Docker.
- Processo: docker login, docker tag e docker push.
- Não há evidência local de push para Docker Hub no repositório.



Evidência visual — reservar espaço para print

Inserir print da imagem publicada ou do docker push bem-sucedido.

Deploy – Docker Compose

DOCKER

- docker-compose.yml orquestra múltiplos serviços.
- Serviços: postgres, pgadmin, app e frontend.
- Rede: app_net.
- Portas: API 8080, frontend 5173 e pgAdmin 5050.

Docker Desktop

- Docker Desktop valida containers, imagens, volumes e logs.
- Containers esperados: sandra-x-andreia, usuario-postgres, usuario-pgadmin e sandra-x-andreia-frontend.



Evidência visual — reservar espaço para print

Inserir print do Docker Desktop com containers rodando.

Apresentação Funcional I

- pgAdmin confirma as tabelas geradas pelo JPA/Hibernate no PostgreSQL.
- Entidades esperadas: usuario, aluno, professor, curso, matricula e assinatura.
- Banco: usuario_db.



Evidência visual — reservar espaço para print

Inserir print do pgAdmin mostrando as tabelas renderizadas.

Apresentação Funcional II

Repositório GitHub: <https://github.com/BPNLeiland/Sandra-X-Andreia>

1

Funcionalidades

Autenticação, CRUD de usuários, alunos, professores, cursos e matrículas.

2

Versionamento

Pode ser apresentado por commits, branches ou cards do Jira.

3

Evidência

Inserir print do Jira SXA ou histórico de commits por sprint.

Bônus – Visão Full-Stack

- Front-end presente em frontend/.
- Telas para aluno, professor e curador.
- frontend/auth.js consome `http://localhost:8080`, incluindo `/aluno` e `/login`.



Docker Compose executa o front-end em container próprio.

Bônus – Integração LLM Gemini

- Não há evidência de integração Gemini/LLM no código atual.
- Evolução proposta: recomendações de cursos, feedback de desempenho ou sugestões de trilha.
- Estratégia: GeminiService consumido por endpoint como /recomendacoes.



Evidência visual — reservar espaço para print

Estratégia: GeminiService consumido por endpoint como /recomendacoes.

Bônus – Testes com Usuários Finais

- Cenário: usuário cadastra aluno, faz login, consulta catálogo, matricula-se e acompanha progresso.
- Feedback esperado: validar clareza do fluxo, facilidade de matrícula e entendimento das recompensas.



Evidência visual — reservar espaço para print

Inserir prints, formulário ou relatos de feedback.

CONSIDERAÇÕES FINAIS

- O projeto aplica separação de camadas, DTOs, JPA, Spring Data, Swagger e Docker.
- A arquitetura favorece manutenção e expansão.
- Pontos fortes: CRUD completo, autenticação, documentação Swagger, front-end e containerização.
- Próximas evoluções: regras completas de gamificação, moedas, vouchers, testes automatizados e Gemini.

Obrigada!